Project Report

Wildlife Information System (WIS)

Document Number: WIS-SRS-001
Version: 2.0
Date: OCT 2026

Prepared by:
Muhammad Ehtisham
Muhammad Abbas
Fahad Khan

Table of Contents

1. INTRODUCTION
- 1.1 Background.....3
- 1.2 Existing System.....3
- 1.3 Issues and Motivation... 4
- 1.4 Problem Statement 4
- 1.5 Scope 4
- 1.6 Project Management 5
- 1.7 Technological Overview 6
- 1.8 Goals & Responsibilities.....6
- 1.9 Process Model 6
- 1.10 Project Management System..... 7

2. SOFTWARE REQUIREMENT SPECIFICATION…….8
- 2.1 Types of Requirements 8
- 2.2 Hardware Requirements..... 8
- 2.3 Software Requirements.....9
- 2.4 Functional Requirements..... 10
- 2.5 Non Functional Requirements 12
- 2.6 Use Cases 13
- 3. ANALYSIS 23
- 3.1 Domain Model.....23
- 3.2 Data Flow Diagram (DFD) 24
- 3.3 System Sequence Diagrams.....26
- 3.4 Activity Diagram 28
- 3.5 Database ER Diagram 29

4. DESIGN 30
- 4.1 Class Diagram 30
- 4.2 Sequence Diagram 31
- 4.3 Architecture.....32
- 4.4 Interface Design 34

5. IMPLEMENTATION.....37
6. TESTING 38
- 6.1 Acceptance Testing.....38
- 6.2 Test Cases.....38
- 6.3 Test Plan.....39

1. INTRODUCTION

1.1 Background

Wildlife means animals that are living naturally in forests, mountains, deserts, local forms and in other environments like elephants, lions, wolves, dogs cows etc. Those animals are very important because they helps to maintain the balance of nature. like predators like as wolves controls the population of other animals, and the birds helps plant to grow by spreading seeds and pollinating flowers. much information about wildlife is collected around the world. This information includes Animal, population number, Movements and migrations routes, Animal's behavior. DNA and genetic information. Health and disease records. However, and There will be real time simulations real time movements its health notification etc. the data is stored in many different places. Different organizations, universities, researchers, and other government departments can easily collect data separately. and There is no single system where all wildlife information can be found. In Pakistan and many other countries, wildlife data is usually stored in reports, spreadsheets, notebooks, and paper records. Because of this, it become difficult for researchers, Doctors, and government and other agencies to access and use the information.

Our System the Wildlife Information System (WIS) is proposed for solving this problem. WIS is a centralized system that can stores all wildlife data in one organized database. Users can easily access and share information through an API, making wildlife research and conservation more efficient. And shows the real time simulation of animals and health records. So Owner can quickly take step.

1.2 Existing System

Currently, the data of wildlife in most countries is managed in these the following ways:

- Government forest departments maintain their own paper base or basic spreadsheets or excel file records of wildlife populations and incidents.
- Researching universities collect the data during research or field trips and store or save it in local databases that are not accessible to the public.
- Other organizations like International as "WWF" and "IUCN" maintain global databases, but they are limited and they have not included local or regional level details.
- And Some IoT based systems which tracks animal movement using GPS collars, but that data kept private and which did not shared in public platforms.

The major weakness of all these approaches is that they do not talk to each other. There is no standard system that collects, organizes, and shares wildlife data openly.

1.3 Issues & Motivation

The following problems motivate the development of the Wildlife Information System:

- Data Isolation: Wildlife data exists in silos. Researchers, forest departments, and NGOs each have their own records but cannot share them.
- No Standard Format: Data is stored in different formats like Excel files, PDF reports, handwritten notes etc. we are making it impossible to combine or compare across organizations.
- Lack of Open Access or Source: Most wildlife databases are closed. Researchers must physically contact organizations and wait weeks to get data they need for their studies.
- Missed Opportunities for Analysis: Because data is fragmented, important patterns such as disease outbreaks, migration route changes, or population decline are often noticed too late.
- Waste of Research Resources: Multiple researchers sometimes collect the same data independently because they do not know it already exists elsewhere. Which waste their time.

1.4 Problem Statement

There is no single system in Pakistan where all the wildlife information is stored or shared. The different universities, research institutions and government institutions have but they keep their own records separately or private, because of this. It is most difficult to find information about animals like their populations, their behavior, DNA and Their Health. This creates many problems Researchers may collect same data repeatedly because they don't know it already exists. And also, the most important information can be missed, and govt agencies may take decisions without complete or accurate data.
The Wildlife Information system solves these problems. It will store wildlife information in one central database. And Authorized users can access this information through secure API and they can use it for research, wildlife monitoring, conservation projects and they can use for decision making.

1.5 Scope

The Wildlife Information System will cover the following:

- It will keep all the wildlife information in one central database. The information will include like animal profile, Population, movement tracking, DNA records, health, their behavior and environment data.
- There will be a backend system that will validate, organize, and manages all incoming data from different sources, like field researchers, IoT tracking devices, and lab results.
- It will include an API. A RESTful API layer that allows external users and applications to send queries and receive structured wildlife data in standard format like JSON.
- The User roles include Data Entry Operators, Researchers, Government Officials, and System Administrators, each will have different level of access.
- There will be a basic web interface in which can search and can enter data manual.

The following are explicitly outside the scope of WIS:

- This system will not install GPS collar or tracking device, manufacturing, or design. It will only store data transmitted from existing devices.
- Real-time live animal tracking maps or animated movement visualization on a geographic interface (this may be a future Phase 2 extension).
- This system will not provide any real time data or tracking or animated maps showing animal movements. But this may be a future plan in version 2.0 extension
- This system will not provide any mobile application for android or iOS. But the other developers may build these separately using the WIS public API.
- This system will not use AI or ML for identifying animal from images or sensor data. This system stores records but does not perform intelligent pattern recognition.
- This system will not connect directly with international database like IUCN Red list or Cities in v1.0 but may connect in later versions.
- This system will not include financial, HR, payroll, or administrative management features. It's mainly focuses on ecological and animal data.
- This system will not include offline data collection tools or special devices for field use in remote areas where the internet is not available.
- This system will not provide automatic alerts for diseases outbreaks, environmental risks or illegal hunting etc.
- This system will not support multiple languages. this system will operate in English only in v1.0.
- The system will not handle legal reporting wildlife permits or enforcement tracking systems.

1.6 Project Management

This project will be managed with a structured approach and everything is clearly defined in phases, responsibilities, and timelines.

Phase	Activity	Duration
Phase 1	Requirements gathering and analysis	2 weeks
Phase 2	System design (DB schema, API design)	2 weeks
Phase 3	Backend development and database setup	3 weeks
Phase 4	API development and testing	2 weeks
Phase 5	Documentation and final review/Testing	1 week

We will use the tools like GitHub for version control, WhatsApp and Physical or online meeting for team communication, and Google Docs for shared documentation. Weekly meetings every Saturday to check progress. And discuss.

1.7 Technological Overview

Component	Technology
Backend	Nodejs with Expressjs
Database	PostgreSQL
API Type	RESTful API (JSON responses)
Authentication	JWT (JSON Web Tokens)
Frontend (basic)	HTML, CSS(Tailwindcss), JavaScript, Reactjs
Version Control	Git / GitHub
Hosting (future)	Cloud (AWS or DigitalOcean)
Data Format	JSON and CSV for exports

1.8 Goals & Responsibilities

Primary Goal: To build a centralized wildlife information database that is accurate, and do simulation to view real time movements and health status, well organized, and accessible through an API for research and analysis purposes.

Secondary Goal: Encourage data sharing among wildlife organizations. and we will also provide a platform that supports conservation and research decision making easily. Also we will implement and build a scalable foundation that can grow with more data and users over time. Which can handle more user at a time.

.

1.9 Process Model

we will use DevOps practices for development, testing, integration and deployment. DevOps combines two thing development and operations for improve collaboration, automation, and repetitive tasks. And for maintain the software quality and ensure system reliability, Version control, automated testing, continuous integration and deployment practices will be used. To manage code changes maintain stability. Also, we will use Model View Controller (MVC) architectural pattern for organize. Models manage app data and business logic. The View will provide UI where user Interact. And the controller will handle user request and coordinate communication between the model and view. Separate parts will make the system easier to develop, testing, and maintain and scaling/extend.

1.10 Project Management System

The team will use GitHub Projects to manage the work of the Project. All Tasks will be organized in a board format with three columns:

1. **To Do:** The tasks that need to be started
2. **In Progress:** The tasks that are currently being worked on.
3. **Done:** The tasks that have been completed

Each team member is assigned specific issues or tasks on the GitHub repository. Which will help us in tracking progress and make sure every task is completed on Time. And later then discuss what each member did.

Wildlife Information System

(WIS)

Software Requirements and Use Cases

Document No: WIS-SRS-003

Version 3.0

October 2026

Prepared by

Muhammad Ehtisham

Fahad Khan

Muhammad Abbas

1. Functional Requirements

Each line below says one thing the system must do. .The IDs are there so we can point to them during testing.

1.1 Login and Access Control

ID	Requirement
FR-01	The system shall let a user log in with email and password.
FR-02	The system shall return a JWT that holds the user'.s role and expires after a set time.
FR-03	The system shall show each role only its own menu items and pages.
FR-04	The system shall block API calls that have no valid token or API key (HTTP 401) and calls the role is not allowed to make (HTTP 403).
FR-05	The system shall let a logged-in user switch to a different role view only if they have that role assigned.
FR-06	The system shall store passwords as bcrypt hashes with a salt.

1.2 Animal Profiles

ID	Requirement
FR-07	The Admin shall be able to add, edit and delete animal records.
FR-08	Each animal shall get a unique ID automatically, made from the species and a number, for example COW-001 or TIGER-001.
FR-09	An animal record shall hold species, name, breed, gender, age, weight, zone and tag ID.
FR-10	The system shall check all fields before saving and return clear field errors if something is missing or wrong.
FR-11	The animals page shall list all animals in a table with ID, species, name, zone, health, activity, temperature and heart rate.
FR-12	The animals table . shall be filterable by species.
FR-13	Clicking an animal in the table shall open its details on the dashboard.
FR-14	The top search box shall .. find an animal by ID, name or species.

1.3 Live Map and Simulation

ID	Requirement
FR-15	The dashboard shall show a satellite map with the zones drawn as coloured rectangles with names.
FR-16	Each animal .. shall appear on the map as an icon with its ID label and a coloured dot for its health status.
FR-17	Hovering over an animal shall show a short card with species, zone, health, activity, temperature and heart rate.
FR-18	Clicking an animal shall select it and show its details in the side panel.
FR-19	The map shall support zoom in, zoom out, fit all animals and fullscreen.
FR-20	The simulation engine shall move each animal toward a random target point inside its own zone and never outside it.

ID	Requirement
FR-21	The simulation engine shall change each animal's behavour from time to time (for example grazing, walking, resting, drinking, stalking) using behavours that suit the species.
FR-22	The simulation engine shall keep temperature between 37.2 and 40.5 C and heart rate between 45 and 120 bpm, changng them a little at each update.
FR-23	The system shall update animal data every 10 seconds by default. The interval shall be changeable between 5 and 60 secnds.
FR-24	The dashboard shall also show recent activity, a 24 hour activity chart and a zone overvie w with animal counts.

1.4 Zones

ID	Requirement
FR-25	The Admin shall be able to add a zone with a name, color and boundary (north, south, east and west values).
FR-26	The Admin shall be able to draw the zone boundary directly on the map instead of typing values.
FR-27	New zones shall apear on the map, in the zone overview and in zone drop-down lists.

1.5 Movement and History

ID	Requirement
FR-28	The system shall save each animal's position with a timestamp at regular steps.
FR-29	The animal panel shall have a History ta b with the distance travelled, number of points, a small route map and an activity timeline.
FR-30	A user shall be able to replay an animal's route on the map and stop the replay.
FR-31	A user shall be able to show the animal's trail on the map for a short time.
FR-32	A user shall be able to download an animal.s history as a CSV file.
FR-33	Data Entry through CSV upload shall remain available. The system shall check evry row, save valid rows, skip bad rows and show a report of what was skipped and why.

1.6 Health and Veterinary Records

ID	Requirement
FR-34	The system shall show each animal's temperature, heart rate, hlth score and last vet check.
FR-35	The system shall show temperature and heart rate trend charts for the selected animal.
FR-36	The system shall set the health status automatically: Healthy up to 39.5C, Warning above 39.5C, Critical above 39.9 C.
FR-37	A Veternarian or Admin shall be able to add and edit vaccination, diagnosis and treatment records. Each record needs a date and a short summary.
FR-38	The Records tab shall list vaccination and medical records with date and status.
FR-39	The system shall keep DNA sample records linked to each animal (sample ID, collection date, lab name, result and species confirmed flag).

1.7 Devices and Sensors

ID	Requirement
FR-40	The syste m shall list devices (GPS collars, health sensors, camera traps, environmental sensors) with ID, linked animal, type, battery, signal, status and last seen time.
FR-41	The page shall show totals for all devices, online devices, devices with warnings and devices with low battery.
FR-42	An Admin or Field Officer shall be able to add a device and link it to an animal.
FR-43	A device shall be marked Low Battery when its battery falls to 20 percent or less.

1.8 Incidents, Alerts and Notifications

ID	Requirement
FR-44	A user with access shall be able to report an incident with a type, related animal, zone and description.
FR-45	Incident types shall include injured animal, anmal missing, abnormal behaviour, human-wildlife conflict, sensor failure and environmental hazard.
FR-46	The incidents page shall list incidents with status Open or Resolved and show totals for this month.
FR-47	A user shall be able to open an incident, read its details and mark it as resolved.
FR-48	The system shall create a notification when an animal's health status becomes Warning or Critical or when a device goes offline.
FR-49	The notification bell shall show the number of unread items and a list of the latest ones, with a Mark all read option.

1.9 Field Tasks and Patrol

ID	Requirement
FR-50	A Field Officer shall see a list of today's tasks with title, zone, pri.ority, due time and done status.
FR-51	A Field Officer shall be able to add a task and tick or untick it as done. Totals for completed and pending tasks shall update at once.
FR-52	The field tasks page shall show an urgent alert card for the most serious open alert, with a Respond button.
FR-53	A Field Officer shall be able to view the day's patrol route on a map with start, checkpoints and end point.
FR-54	A Field Officer shall be able to change the patrol route by clicking points on the map, and start the patrol.

1.10 Reports and Analytics

ID	Requirement
FR-55	The reports page shall show total animals, healthy animals, average temperature and average heart rate.
FR-56	The reports page shall show charts for species distribution, health distribution and animals per zone.
FR-57	The report numbers shall refresh while the .simulation is running.

1.11 Search, Export and API Access

ID	Requirement
FR-58	The system shall s earch animals by species, location or date range and show results in pages of 20.
FR-59	Researchers shall be able to download movement data, health records, activity log, a complete dataset and an analytics summary as CSV, and a full export as JSON.
FR-60	The researchr page shall show the user's API key with Copy and Regenerate buttons.
FR-61	The researcher page shall have a data explorer where the user picks an endpoint, adds query parameters and sees the JSON or CSV response.
FR-62	The system shall show API usage: requests today, rate limit, data points available and the time of the last export.
FR-63	The API shall return JSON, use standard HTTP status codes and require a JWT or API key on every request except login.
FR-64	The system shall log every API request made with an API key.

1.12 User Management and Settings

ID	Requirement
FR-65	The Admin shall be able to add, edit, deactivate and delete users and assign one of the roles.
FR-66	The system shall make sure e ach user has a unique email address.
FR-67	The users page shall show name, email, role, status and last login, along with the role permission table.
FR-68	The Admin shall be able to start and pause the simulation, set the update interval, switch animal movement, health simulation and alerts on or off, and set the movement speed (slow, normal, fast).
FR-69	The Admin shall be able to reset the simulation to its starting data and export all data.
FR-70	The Admin shall be able to see and generate API keys and see recent webhook events.

2. Non-Functional Requirements

ID	Type	Requirement
NFR-01	Performance	Standard API requests shall respond within 2 seconds.
NFR-02	Performance	The live map shall keep moving smoothly with at least 50 animals on screen.
NFR-03	Scalability	The database shall handle up to 1 million ..animal and movement records without noticeable slowdown.
NFR-04	Security	All API calls shall use HTTPS and require JWT or API key authentication.
NFR-05	Security	Passwords shall be hashed with b crypt and salt. API keys shall be shown in full only when created.
NFR-06	Availability	The system shall be available 99 percent of the time after deployment.
NFR-07	Data integrity	Every record shall have created and updated timestamps, and the system shall log who created or changed it.
NFR-08	Usability	A new user shall be able to use the main pages for their role without training.

ID	Type	Requirement
NFR-09	Compatibility	The web app shall work in current Chrome, Firefox and Edge, on desktop and tablet screens.
NFR-10	Compatibility	The API shall follow REST rules so any language or tool can use it.

3. Use Cases

3.1 Actors

Actor	Description
Admin	Runs the whole system and manages users, data and settings.
Field Officer	Works in the field and uses tasks, patrols and incidents.
Researcher	Studies the data and exports it.
Veterinarian	Looks after health and vaccination records.
External App (API user)	Another program that reads data using an API key.
Simulation Engine	Background service on the server that creates live animal data. It has no screen.

3.2 Use Case Summary

ID	Name	Main actor	Category
UC-01	Login and Logout	All users	Access
UC-02	View Live Map and Animal Details	Admin, Field Officer, Researcher, Vet	Monitoring
UC-03	Manage Animal Records	Admin	Data management
UC-04	View Movement History and Replay Route	Admin, Field Officer, Researcher	Monitoring
UC-05	Manage Zones	Admin	Data management
UC-06	Control Simulation	Admin	Simulation
UC-07	Manage Health Records	Veterinarian, Admin	Data management
UC-08	Manage Devices and Sensors	Admin, Field Officer	Devices
UC-09	Report and Resolve Incident	Field Officer, Admin	Field work
UC-10	Manage Field Tasks	Field Officer	Field work
UC-11	Plan and Start Patrol Route	Field Officer	Field work
UC-12	View Reports and Analytics	Researcher, Vet, Admin	Reports
UC-13	Export Data	Researcher, Vet, Admin	Reports
UC-14	Access Data Through API	Researcher, External App	API
UC-15	Manage Users and Roles	Admin	User management
UC-16	View Notifications and Alerts	All logged-in users	Monitoring

3.3 Use Case Diagram

This diagram shows every role and what it can do. Admin could do everything the others can, but to keep the picture readable Admin is only linked to the admin tasks.



Figure 2: Use case diagram for WIS

3.4 Use Case Diagrams by Role

Admin

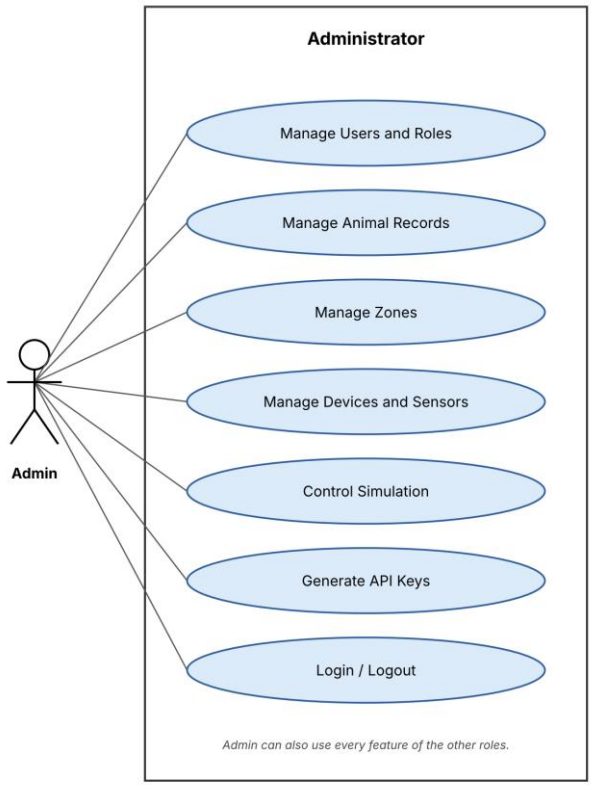


Figure 3: Admin use cases

Field Officer

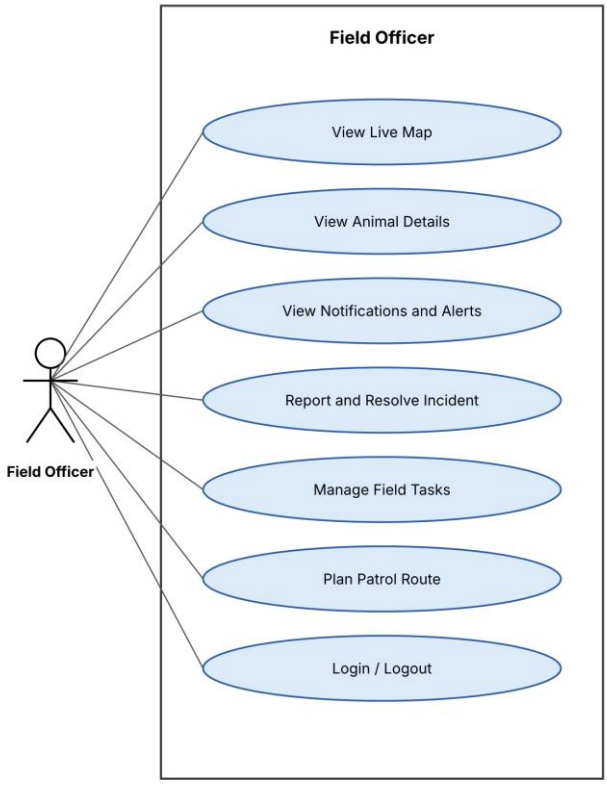


Figure 4: Field Officer use cases

Researcher and API User

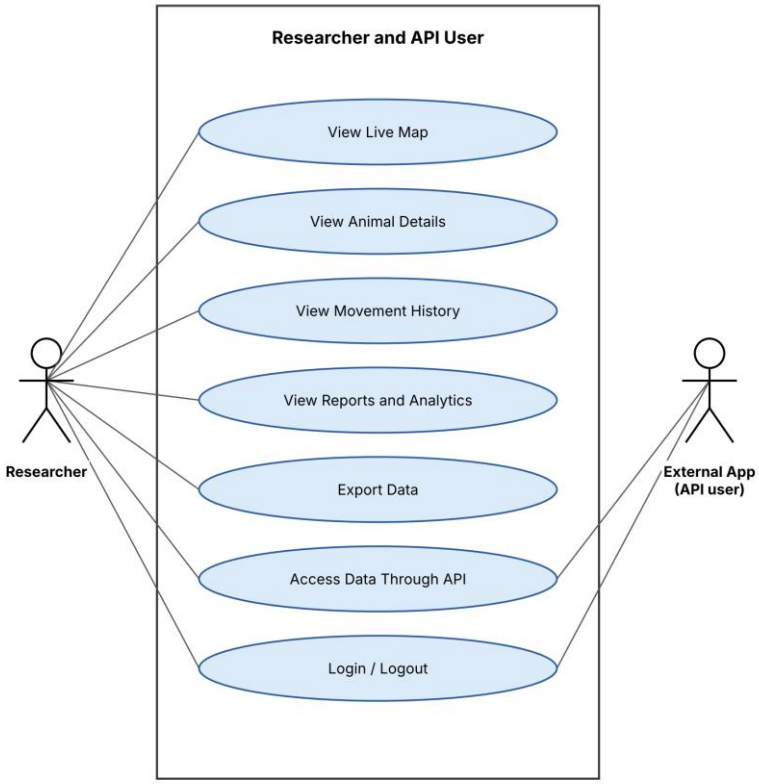


Figure 5: Researcher and API user use cases

Veterinarian

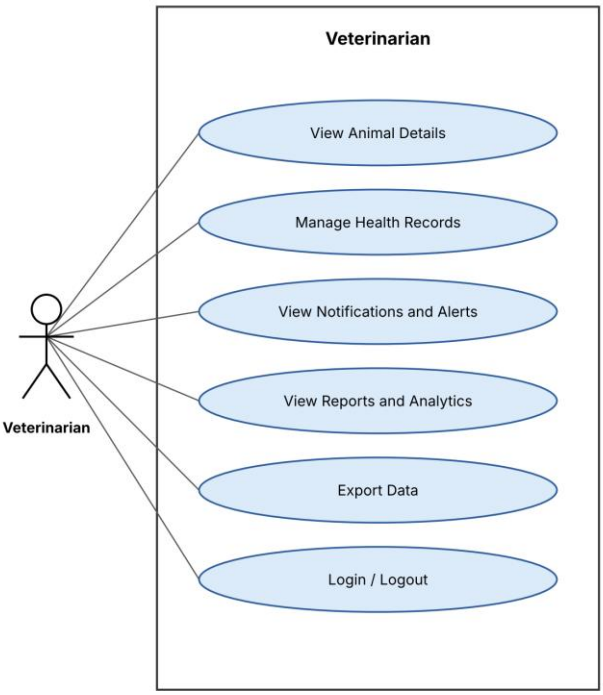


Figure 6: Veterinarian use cases

3.5 Use Case Descriptions

UC-01: Login and Logout

Actor	All users
Pre-condition	The user has an account and it is active.
Post-condition	The user is logged in and sees the menu and default page for their role. On logout the token is removed.
Main Flow	1. User opens the login page and enters email and password. 2. System finds the user by email and compares the password with the bcrypt hash. 3. System creates a JWT that holds the user's role. 4. System op ens the default page for the role (Admin: Dashboard, Researcher: Researcher API, Field Officer: Field Tasks).
Alternate Flow	Wrong email or password: HTTP 401 with a general message that doesn't say which one was wrong. Deactivated account: HTTP 403.

UC-02: View Live Map and Animal Details

Actor	Admin, Field Officer, Researcher, Veterinarian
Pre-condition	User is logged in.
Post-condition	The user sees animals on the map and the full details of the chosen animal.
Main Flow	1. User opens the Dashboard or Map page. 2. System shows the map, zones and animals at their current positions. 3. User hovers .over an animal to see the short info card. 4. User clicks the animal. 5. System opens the side panel with Overview, Health, Records, Location and History tabs. 6. System keeps updating temperature, heart rate and position every update interval.
Alternate Flow	Map tiles fail to load: the system still shows animals and zones and shows a message. Animal not found: HTTP 404.

UC-03: Manage Animal Records

Actor	Admin
Pre-condition	User is logged in as Admin.
Post-condition	An animal is added, ..updated or deleted. A new animal has a unique ID.
Main Flow	1. Admin opens the Animals page and clicks Add Animal. 2. Admin enters species, name, breed, gender, age, weight and zone. 3. System checks all fields. 4. System creates the ID (for example COW-002), saves the animal and places it inside its zone. 5. System shows a success message and the animal appears on the map and in the table.
Alternate Flow	Missing or invalid fields: HTTP 400 with a message for each field. Deleting an animal asks for confirmation first.

UC-04: View Movement History and Replay Route

Actor	Admin, Field Officer, Researcher
Pre-condition	User is logged in. The animal has saved positions.
Post-condition	The user has seen the route, or downloaded the history as CSV.
Main Flow	1. User selects an animal and opens the History tab. 2. System shows distance, number of points, route map and activity timeline. 3. User clicks Play @ Route. 4. System moves a marker along the saved path. 5. User clicks Download to get the history as a CSV file.
Alternate Flow	Fewer than two saved points : system shows the message 'Not enough trail data'.

UC-05: Manage Zones

Actor	Admin
Pre-condition	User is logged in as Admin.
Post-condition	A new zone is saved and shown on all maps and drop-down lists.
Main Flow	1. Admin clicks Add Zone on the map. 2. Admin enters a name, color and boundary values, or draws the boundary on the map by clicking points and double-clicking to finish. 3. Admin saves the zone. 4. System checks that the name is filled and the boundary is valid. 5. System adds the zone to the map, zone overview and forms.
Alternate Flow	No name or fewer than 3 drawn points : system shows a warning and does not save. Admin can cancel drawing at any time.

UC-06: Control Simulation

Actor	Admin
Pre-condition	User is logged in as Admin.
Post-condition	The simulation runs with the new settings.
Main Flow	1. Admin opens Settings. 2. Admin switches the simulation on or off, picks the update interval and speed, and switches movement, health simulation and alerts. 3. System saves the settings and applies them to the simulation engine. 4. Admin can press @ Reset Simulation to go back to the starting animals.
Alternate Flow	Reset asks for confirmation. If the engine cannot be reached, system shows an error and keeps the old settings.

UC-07: Manage Health Records

Actor	Veterinarian, Admin
Pre-condition	User is logged in as Veterinarian or Admin. The animal exists.
Post-condition	A vaccination, diagnosis or treatment record is saved for the animal.
Main Flow	1. User opens the animal and goes to the Records tab. 2. User clicks Add Record and fills in type, date and a short summary. 3. System checks the fields. 4. System saves the record and shows it in the list with its status.
Alternate Flow	Missing date or summary: HTTP400. Animal ID not found: HTTP404. Other roles cannot add records: HTTP403.

UC-08: Manage Devices and Sensors

Actor	Admin, Field Officer
Pre-condition	User is logged in as Admin or Field Officer.
Post-condition	A device is added or its latest status is shown.
Main Flow	1. User opens Devices and Sensors. 2. System shows totals and the device table with battery and signal. 3. User clicks Add Device and enters device ID, type and animal. 4. System saves the device and shows it online.
Alternate Flow	Device ID already exists : HTTP 409. Battery 20 percent or lower: status shows Low Battery.

UC-09: Report and Resolve Incident

Actor	Field Officer, Admin
Pre-condition	User is logged in as Field Officer or Admin.
Post-condition	The incident is saved as Open, or changed to Resolved.
Main Flow	1. User opens Incidents and clicks Re port Incident. 2. User picks the type, animal and zone and writes a description. 3. System saves the incident as Open and adds it to the table. 4. Later, user opens the incident and clicks Resolve. 5. System sets the status to Resolved and updates the totals.
Alternate Flow	Empty description: system asks for one and does not save.

UC-10: Manage Field Tasks

Actor	Field Officer
Pre-condition	User is logged in as Field Officer.
Post-condition	Tasks are added or marked done, and the totals are updated.
Main Flow	1. Officer opens Field Tasks and sees the urgent alert, totals and task list. 2. Officer clicks Add Task and enters title, zone, priority and due time. 3. System saves the task. 4. Officer ticks a task when it is finished. 5. System updates the completed and pending counts.
Alternate Flow	Empty title:: system asks for a title. Officer can untick a task to reopen it.

UC-11: Plan and Start Patrol Route

Actor	Field Officer
Pre-condition	User is logged in as Field Officer.
Post-condition	The patrol route is saved and the patrol is marked active.
Main Flow	1. Officer opens Patrol Route and sees the current route with start, checkpoints and end. 2. Officer clicks Edit Route and clicks points on the map. A double-click finishes the route. 3. System draws and saves the ne.w route. 4. Officer clicks Start Patrol. 5. System marks the patrol as active.
Alternate Flow	Fewer than 2 points .: system asks for at least 2 points. Officer can cancel editing and keep the old route.

UC-12: View Reports and Analytics

Actor	Researcher, Veterinarian, Admin
Pre-condition	User is logged in with one of these roles.
Post-condition	The user has seen the current numbers and charts.
Main Flow	1. User opens Reports or Analytics. 2. System shows total animals, healthy count, average temperature and average heart rate. 3. System draws species, health and zone charts. 4. Numbers refresh while the simulation runs.
Alternate Flow	No animals in the system: charts are empty and totals show 0.

UC-13: Export Data

Actor	Researcher, Veterinarian, Admin
Pre-condition	User is logged in with one of these roles.
Post-condition	A CSV or JSON file is downloaded.
Main Flow	1. User opens the download area and picks a dataset (movement, health, activity, complete dataset, analytics or JSON). 2. System collects the data with the current filters. 3. System builds the file. 4. Browser downloads the file.
Alternate Flow	No matching records : system gives a file with only headers and tells the user.

UC-14: Access Data Through API

Actor	Researcher, External App
Pre-condition	The caller has a valid JWT or API key.
Post-condition	The caller gets a JSON (or CSV) response with status 200.
Main Flow	1. Caller sends a GET? request, for example /api/animals?species=Tiger, with the token or key. 2. System checks the token or key and the role. 3. System runs the query. 4. System returns the result as JSON. 5. System counts the request in the usage numbers.
Alternate Flow	Invalid or expired token or key : ?401. Role not allowed: ?403. Rate limit passed: 429. No results: ?200 with an empty list.

UC-15: Manage Users and Roles

Actor	Admin
Pre-condition	User is logged in as Admin.
Post-condition	A user is created, changed, deactivated or deleted.
Main Flow	1. Admin opens Users and Roles and clicks Add User. 2. Admin enters name, email and role. 3. System checks that the email is not used. 4. System saves the user and creates a password hash. 5. User appears in the table. Admin can also edit, deactivate or delete a user.
Alternate Flow	Duplicate email : “'Email already exists'. Deleting a user asks for confirmation.

UC-16: View Notifications and Alerts

Actor	All logged-in users
Pre-condition	User is logged in.
Post-condition	The user has read the notifications and the unread count is updated.
Main Flow	1. The simulation engine finds an animal with Warning or Critical health, or a device that went offline. 2. System creates a notification. 3. The bell shows the unread count. 4. User opens the bell & reads the list. 5. User clicks Mark all read.
Alternate Flow	No notifications : the list shows that nothing new has come in.

Wildlife Information System (WIS): Analysis and Design

Document No. WIS-SRS-001 · Version 3.0 · October 2026

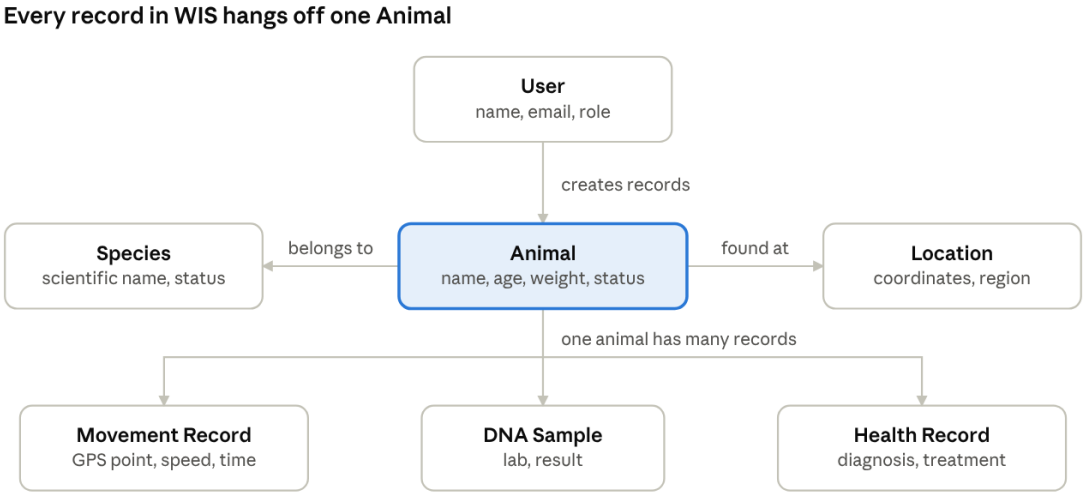
Prepared by: Muhammad Ehtisham, Muhammad Abbas, Fahad Khan

3. Analysis

This chapter analyses the problem described in Chapter 1: wildlife data is scattered across universities, forest departments and NGOs in paper records, spreadsheets and private databases. WIS stores this data in one central PostgreSQL database and exposes it through a secure RESTful API (JSON) and a basic web interface. The four user roles are Data Entry Operator, Researcher, Government Official and System Administrator. Data arrives from field researchers, IoT tracking devices and lab results. The items Chapter 1 puts out of scope (live tracking maps, automatic alerts, AI/ML, mobile apps, multiple languages) are not analysed here.

3.1 Domain Model

The domain model shows the real-world concepts WIS must record and how they relate, independent of any technology. The Animal is the central concept: it belongs to one Species, is found at a Location, and accumulates Movement Records, DNA Samples and Health Records over time. Users create these records.



domain model · 7 concepts

Users create records; each animal links to one species and one location and owns many movement, DNA and health records.

Domain concepts

Concept	Description	Key attributes
Species	A kind of animal, for example Snow Leopard	common name, scientific name, classification, conservation status, description
Animal	An individual animal tracked by the system	name, gender, date of birth / age, weight (kg), unique marking, status (alive/dead)
Location	A place where animals are found or recorded	name, latitude, longitude, altitude, region, country, habitat type
Movement Record	One GPS reading for an animal	latitude, longitude, altitude, speed (km/h), recorded time, source (for example GPS collar)
DNA Sample	A genetic sample analysed by a lab	sample ID, collection date, sample type, lab name, lab result, species confirmed
Health Record	A veterinary event for an animal	record date, record type, diagnosis, treatment, veterinarian, next follow-up date
User	A person who logs in to WIS	full name, email, role, organization, status

Relationships

From	Relationship	To	Multiplicity
Animal	belongs to	Species	many animals to one species
Animal	is found at	Location	many animals to one location
Animal	has	Movement Record	one animal to zero or more records
Animal	has	DNA Sample	one animal to zero or more samples
Animal	has	Health Record	one animal to zero or more records
User	creates	Movement, DNA and Health records, and animals	one user to many records

Proposed access by role (to be confirmed with stakeholders)

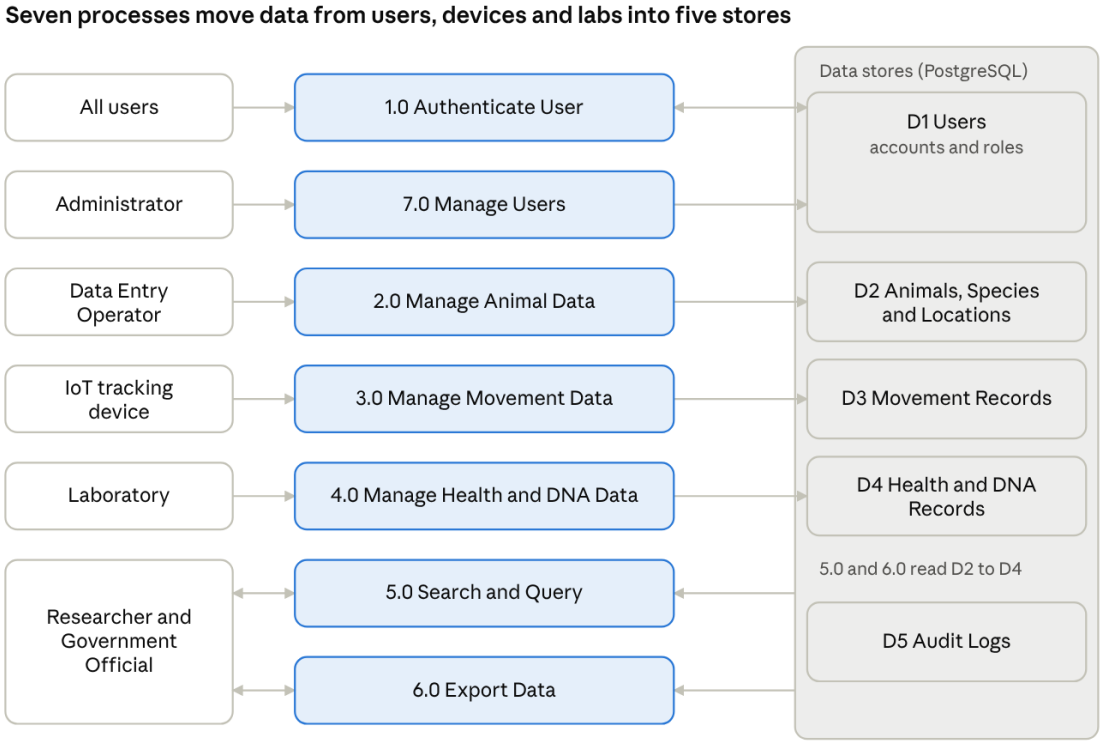
Role	Access
Data Entry Operator	Create and update animal, movement, health and DNA records
Researcher	Search, view and export data through the web interface and API
Government Official	Search, view and export data; view dashboard statistics
System Administrator	Full access, plus user management

3.2 Data Flow Diagram (DFD)

Level 0 (context). WIS is a single process that exchanges data with six external entities.

External entity	Sends to WIS	Receives from WIS
Data Entry Operator	Animal, species, location, health and DNA data; CSV files	Confirmation or validation errors
Researcher	Search filters, API requests, export requests	Result lists, JSON and CSV data
Government Official	Search filters, report requests	Result lists, dashboard statistics, exports
System Administrator	User account details	User list, confirmations
IoT tracking device / field system	GPS readings (source: GPS collar)	Acknowledgement
Laboratory	DNA sample results	Acknowledgement

Level 1. The system is decomposed into seven processes and five data stores.



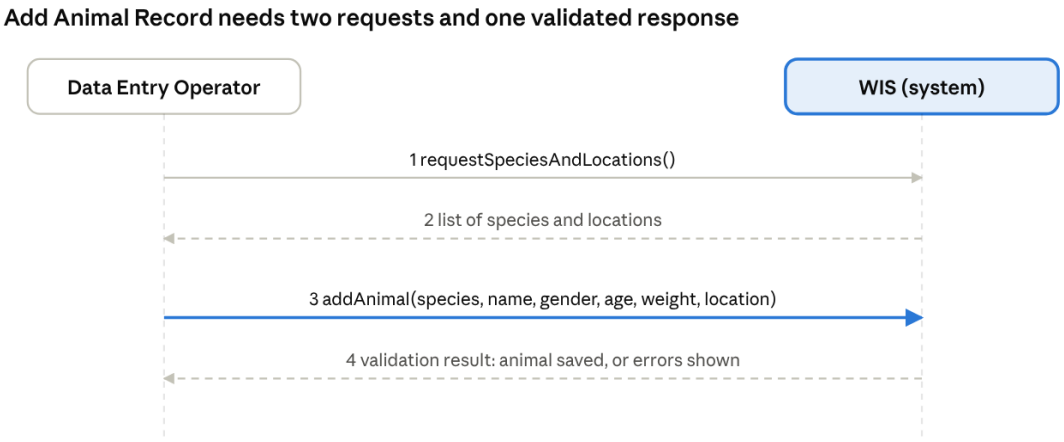
Level 1 DFD · 7 processes, 6 entities, 5 stores

Process	Purpose
1.0 Authenticate User	Checks email and password, issues a JWT, verifies the token and role on every request
2.0 Manage Animal Data	Adds, updates, deletes and views animals, species and locations
3.0 Manage Movement Data	Accepts GPS data from devices and bulk CSV uploads, validates and stores it
4.0 Manage Health and DNA Data	Stores veterinary records and lab DNA results
5.0 Search and Query	Handles filtered searches from the web interface and the REST API
6.0 Export Data	Produces JSON and CSV exports of query results
7.0 Manage Users	Creates, edits and removes user accounts (administrator only)

Data store	Contents
D1 Users	Accounts, password hashes, roles
D2 Animals and Species	Animal profiles, species, locations
D3 Movement Records	GPS readings
D4 Health and DNA Records	Veterinary records and lab DNA results
D5 Audit Logs	Record of who changed what and when

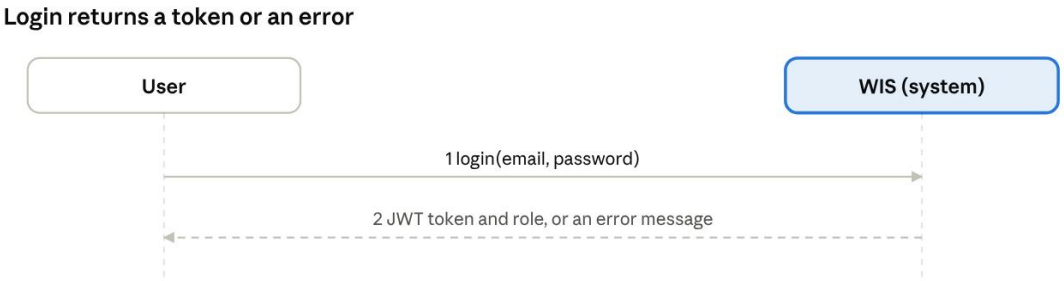
3.3 System Sequence Diagrams

A system sequence diagram treats WIS as a black box and shows only the messages between an actor and the system. The diagram below covers Add Animal Record; the tables cover the other key use cases.



SSD · Add Animal Record · 4 messages

SSD 1: Login



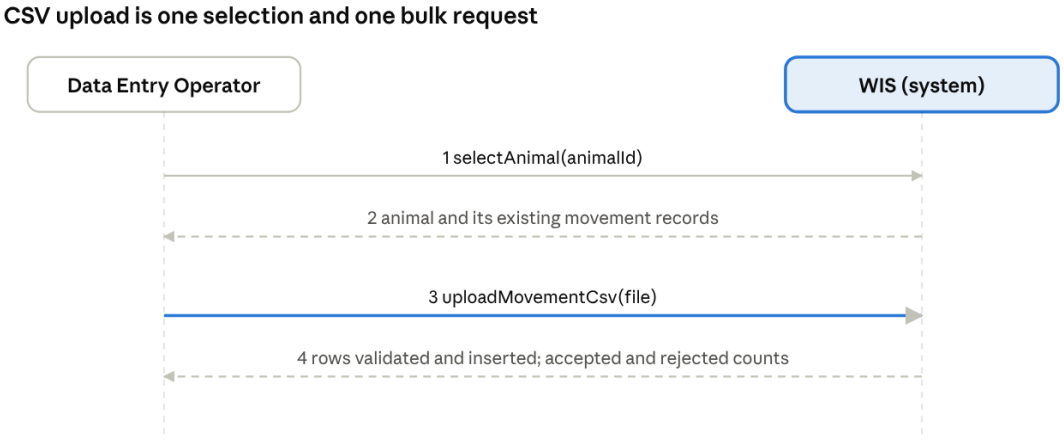
SSD 1 · Login

#	Actor to System	System response
1	login(email, password)	Valid: JWT token and role returned. Invalid: error message

SSD 2: Add Animal Record (diagram above)

#	Actor to System	System response
1	requestSpeciesAndLocations()	List of species and locations
2	addAnimal(species, name, gender, age, weight, location)	Validation result
3	(if valid)	Animal saved, success message with animal ID

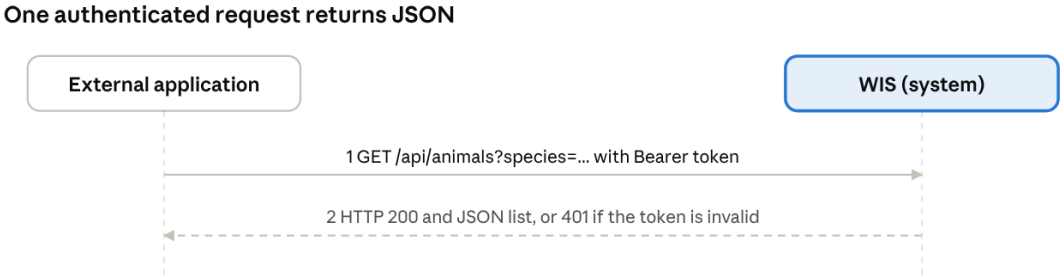
SSD 3: Upload Movement CSV



SSD 3 · Upload Movement CSV

#	Actor to System	System response
1	selectAnimal(animalId)	Animal and existing movement records
2	uploadMovementCsv(file)	Rows validated and inserted; count of accepted and rejected rows

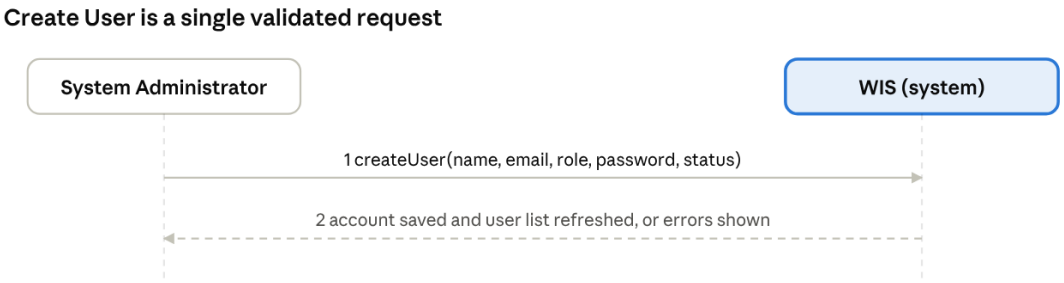
SSD 4: Query Animal Data via API



SSD 4 · Query Animal Data via API

#	Actor to System	System response
1	GET /api/animals?species=... with Bearer token	HTTP 200 and JSON list, or 401 if the token is invalid

SSD 5: Create User (administrator)

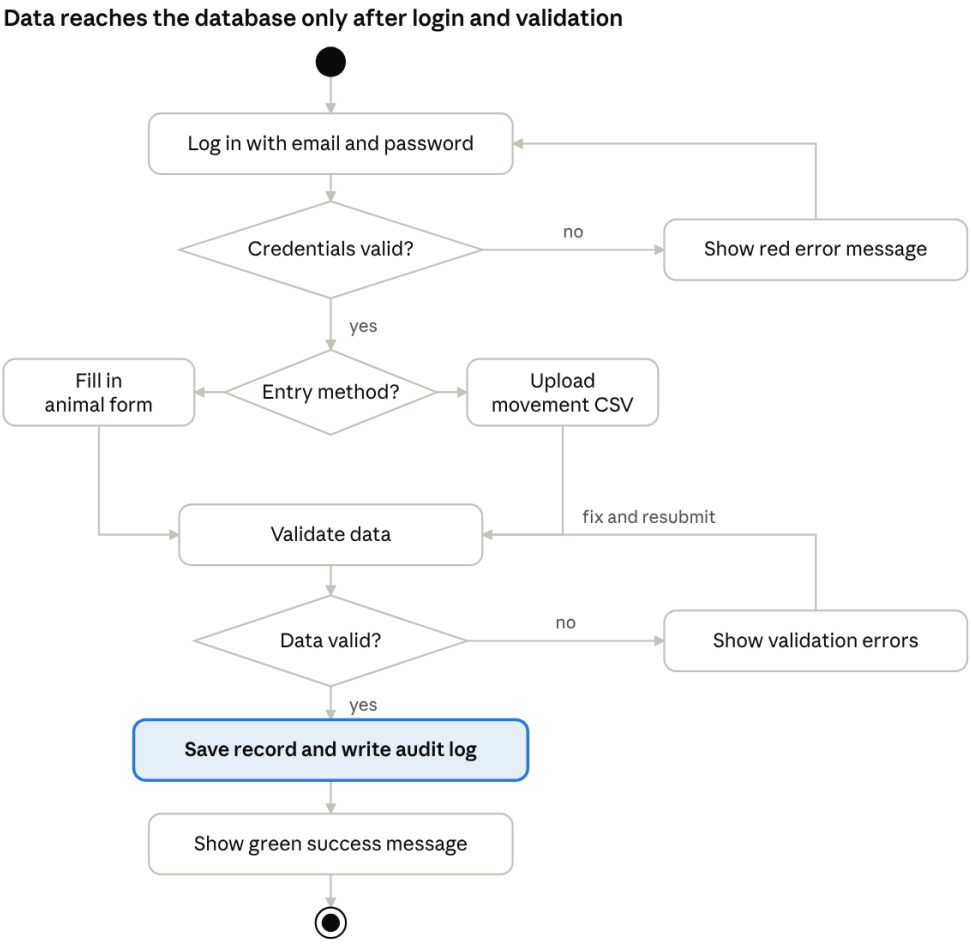


SSD 5 · Create User

#	Actor to System	System response
1	createUser(name, email, role, password, status)	Validation result
2	(if valid)	Account saved, user list refreshed

3.4 Activity Diagram

The activity diagram models the main data-entry workflow: a user logs in, chooses to enter a record manually or upload a CSV file, the system validates the data, and valid data is stored.



activity diagram · 2 decisions, 1 loop each

3.5 Database ER Diagram

The ER diagram maps the domain model to PostgreSQL tables. Primary keys are shown as PK and foreign keys as FK. Each data table also carries a created_by foreign key to users, which supports the audit trail.

↗ ER diagram · 8 tables

Table	Columns
species	species_id (PK), common_name, scientific_name, classification, conservation_status, description
locations	location_id (PK), name, latitude, longitude, altitude, region, country, habitat_type, description
animals	animal_id (PK), species_id (FK), location_id (FK), name, date_of_birth, gender, weight_kg, unique_marking, status, created_by (FK), created_at, updated_at

Table	Columns
movement_records	movement_id (PK), animal_id (FK), location_id (FK), recorded_at, latitude, longitude, altitude, speed_kmh, source, created_at
dna_samples	dna_sample_id (PK), animal_id (FK), sample_id, collection_date, sample_type, lab_name, lab_result, species_confirmed, notes
health_records	health_record_id (PK), animal_id (FK), record_date, record_type, description, veterinarian_name, diagnosis, treatment, next_followup_date, created_at
users	user_id (PK), full_name, email (unique), password_hash, role, organization, status, created_at
audit_logs	log_id (PK), user_id (FK), action, entity, entity_id, logged_at

and success messages in green.

Wildlife Information System (WIS): Analysis and Design

Document No. WIS-SRS-001 · Version 2.0 · October 2026

Prepared by: Muhammad Ehtisham, Muhammad Abbas, Fahad Khan

4. Design

4.1 Class Diagram

The class diagram shows the core classes with their attributes and methods. Animals is the aggregate root; Movement_Record, DNA_Sample and Health_Record each belong to one animal, and Users create them.

↗ class diagram · 7 classes

The tables below list every attribute and method; the figure shows the key ones.

Class	Attributes	Methods
Species	speciesId, commonName, scientificName, classification, conservationStatus, description	addSpecies(), updateSpecies(), deleteSpecies(), getSpeciesById(id)
Animals	animalId, speciesId, locationId, name, dateOfBirth, gender, ageYears, weightKg, uniqueMarking, status, addedBy, createdAt, updatedAt	addAnimal(), updateAnimal(), deleteAnimal(), getAnimalById(id), getAnimalsBySpecies(speciesId), getMovementHistory(), getHealthRecords(), getDNASamples()
Locations	locationId, name, latitude, longitude, altitude, region, country, habitatType, description	addLocation(), updateLocation(), deleteLocation(), getLocationById(id)
Movement_Record	movementId, animalId, locationId, recordedAt, latitude, longitude, altitude, speedKmh, source, createdAt	addMovement(), getMovementByAnimal(animalId), getMovementByDateRange(animalId, start, end), deleteMovement(id), bulkInsert()
DNA_Sample	dnaSampleId, animalId, sampleId, collectionDate, sampleType, labName, labResult, speciesConfirmed, notes	addDnaSample(), getDnaSamplesByAnimal(animalId), updateDnaSample(), deleteDnaSample(id)
Health_Record	healthRecordId, animalId, recordDate, recordType, description,	addAnimalRecord(), getHealthByAnimal(animalId),

Class	Attributes	Methods
	veterinarianName, diagnosis, treatment, nextFollowupDate, createdAt	updateHealthRecord(), deleteHealthRecord(id)
Users	userId, fullName, email, passwordHash, role, organization, status, createdAt	register(), login(email, password), logout(), updateProfile(), changePassword()

4.2 Sequence Diagram

The sequence below shows how an external application queries animal data through the API: GET /api/animals?species=Snow+Leopard with an Authorization: Bearer token.

↗ sequence diagram · 6 participants, 9 messages

1. The external app sends the GET request with its JWT in the Authorization header.
2. The API Gateway asks the Authentication Service to validate the token.
3. The Authentication Service confirms the token is valid and returns the user role.
4. The API Gateway forwards the request with its parameters to the Query Controller.
5. The Query Controller passes the parameters to the Animal Service, which constructs the SQL query joining the animals and species tables.
6. The Animal Service queries the database for the relevant rows.
7. The database returns the query result.
8. The Animal Service returns the result as JSON to the Query Controller.
9. The API Gateway responds to the external app with HTTP 200 and the JSON payload.

If step 3 fails, the gateway stops the request and responds with HTTP 401.

4.3 Architecture

WIS follows a 3-tier architecture. External users and applications reach the Application Layer through the RESTful API; the Application Layer handles authentication, business logic and data formatting; the Data Layer stores everything persistently.

↗ 3-tier architecture · 3 layers

Layer	Component	Responsibility
Presentation	HTML/CSS/JavaScript web interface	Data entry, search and export
Application	Node.js + Express.js API server, JWT authentication	Business logic, authentication, query processing
Data	PostgreSQL	Wildlife records, users and audit logs

Key API endpoints

Method	Endpoint	Description
GET	/api/animals	List animals (with filters)
GET	/api/animals/:id	Get one animal by ID
POST	/api/animals	Add an animal record
PUT	/api/animals/:id	Update an animal record
DELETE	/api/animals/:id	Delete an animal record
GET	/api/animals/:id/movements	Movement history for an animal
POST	/api/movements/upload	Bulk upload CSV movement data
GET	/api/animals/:id/health	Health records for an animal
GET	/api/animals/:id/dna	DNA records for an animal
POST	/api/auth/login	Log in and receive a JWT
GET	/api/species	List all species

4.4 Interface Design

WIS has an easy-to-use web interface with a sidebar for Dashboard, Animals, Movements, Health, DNA Samples, Users, Reports and Logout.

Page	Contents
Login	Email field, password field, Remember me, Forgot password link, Login button; shows a red error message for incorrect credentials
Dashboard	Welcome message, statistic cards (total animals, species, movements, health records), a recent-records table and a View All button
Animal List	Searchable, sortable table of all animals with species and location filters, an Add Animal button, paging, and View, Edit and Delete buttons per row
Add Animal Form	Species, name, gender, age (years), weight (kg) and location fields, with Submit and Cancel buttons
Movement Records	Sortable table of GPS coordinates and timestamps for one animal, with an Upload CSV button
API Documentation	Base URL, endpoint list, required parameters and example responses

Page	Contents
User Management (admin only)	Sortable user table (ID, name, email, role, status, created date) with Create User, Edit and Delete; Create and Edit forms with role and status selectors

Design principles: minimal colour, only blue and white; all buttons have descriptive names; errors are shown in red and success messages in green.

5. IMPLEMENTATION

We Will write this document along with development task .

6. TESTING

Testing is an important phase of project which verifies whether the developed or implement system satisfies the specified functional and non-functional requirements. and The purpose of testing is to identify errors, validate system functionality, check access permission like login register etc, and ensure that it CRUD operation is working correctly. Testing will be performed throughout development using a combination like unit testing, integration testing, system testing, and acceptance testing. All valid and invalid inputs will be tested to verify the correct, security, and reliability of the system. So because it will not crash when in production.

6.1 Acceptance Testing

Acceptance testing will verify that the Wildlife Information System meets the requirements defined in this Software Requirements Specification (SRS). The system will be considered acceptable when its major functions operate correctly and the specified acceptance criteria are satisfied. The acceptance criteria are as follows:

- Authorized users should be able to log in successfully and receive tokens like session and login token.
- The system should allow users which have permission to create, retrieve, update, and delete wildlife records.
- The system should validate Data which is newly added before storing it in the PostgreSQL database.
- Authorized users should be able to upload correctly formatted CSV files in which have animal movement information and other related to it.
- The system should validate uploaded CSV files and report or tell error about invalid or incomplete records appropriately.
- Users should be able to search using filter records and search by query, such as animal name and species and geographical information.
- The system should return correct HTTP status code from backend to check whether system is correct working or not. and meaningful error messages for invalid requests, unauthorized access, and records that do not exist. And the message will show in frontend UI.
- The system should have role based access control so that users can perform only the operations permitted by their assigned roles.
- Admin can create user account and assign appropriate roles. And can delete or update information. Basically can control All the system overflow.
- Users should be able to retrieve and export available wildlife information in the supported data formats.
- The system should prevent unauthorized users from accessing private API. And will add rate limiting for prevent brute force attack. and restricted operations.
- The system should store and retrieve correctly without any data loss or corruption or crashed software.

Acceptance testing will be performed before the final submission to confirm that the implemented system satisfies its specified requirements.

6.2 Test Cases

TC ID	Feature	Input	Expected Output
TC-01	User Login	Valid email and password	JWT token returned, status 200
TC-02	User Login (wrong)	Valid email, wrong password	Status 401 Unauthorized
TC-03	Add Animal	Valid animal data fields	Record saved, new ID returned
TC-04	Add Animal (missing)	Species name missing	Status 400, field error message
TC-05	Get Animal by ID	Valid animal ID in URL	Full animal record in JSON
TC-06	Get Animal (wrong ID)	ID that does not exist	Status 404 Not Found
TC-07	Search by Species	species=Snow Leopard query	List of matching animals
TC-08	CSV Upload (valid)	Correctly formatted CSV file	All rows saved successfully
TC-09	CSV Upload (bad)	CSV with missing coordinates	Rows skipped, error report shown
TC-10	API without a token	GET request without Auth header	Status 401 Unauthorized
TC-11	Export Data	Request to export search results	JSON or CSV file downloaded
TC-12	Admin creates user	New user details with role	User account created successfully

6.3 Test Plan

Testing Approach: Black Box Testing

The black box testing approach will be used as the main testing technique by the development team.

Tests are created solely based on input values and expected output results.

Black box testing is appropriate for this case due to the following reasons:

- There are specific inputs and expected outputs of API endpoints.
- Functional requirements will be validated without having to read the code.
- This reflects the way actual users and other applications will work with the product.

Test Levels

Unit Testing: Unit tests will be conducted for each API endpoint with the help of Postman. In each unit test, only one API endpoint is tested.

Integration Testing: Integration testing will be carried out on the completion of development of all the API endpoints, and they will be tested together.

System Testing: System testing will involve conducting tests for the entire application system from logging in to exporting the data via APIs.

Acceptance Testing: The final stage of testing in which involves testing against the requirements specification.

Test Tools

Postman: For manual API testing.

Browser DevTools: For testing the web interface.

Test Schedule

Phase	Test Type	When
After each endpoint	Unit Test (Postman)	During development
After full API build	Integration Test	After Increment 2
Before submission	System & Acceptance Test	Final week

Definitions & Acronyms

Term / Acronym	Definition
WIS	Wildlife Information System
API	Application Programmable Interface
REST	Representational State Transfer
JWT	JSON Web Token
FR	Functional Requirement
NFR	Non-Functional Requirement
UC	Use Case
CRUD	Create, Read, Update, Delete
CSV	Comma Separated Values
JSON	JavaScript Object Notation
DFD	Data Flow Diagram
ER	Entity Relationship